FemboyEngine 0.4.0-alpha.1 — getting started

This document lists what to install and which files must sit next to the engine to run the editor, the runtime and the development tools on Windows, Linux and macOS.

Supported platforms

Platform	Editor	Game runtime	Toolchain
Windows 10/11 x64	yes	yes	clang-cl 23.1.4+ (primary), MSVC 19.44+, GCC 16.2.0+
Linux x64	yes	yes	Clang 23.1.4+, GCC 16.2.0+
macOS 14+ (Apple Silicon and Intel)	yes	yes	AppleClang 21+ (Xcode 26.4+), Metal backend

The editor and the game runtime target the three desktop platforms above. SDL3 supplies the window, input and application lifecycle.

Required software

MSVC 19.44+ is supported as an alternative. On Windows the project builds with clang-cl from LLVM as the reference toolchain, or with MSVC (`cl.exe`) when a team standardizes on it. Both consume the Windows SDK and the MSVC standard library; clang-cl remains what CI and the release gate run.

Tool	Minimum	Validated	Purpose
CMake	3.28	3.31.6	configure and build
Ninja	1.11	1.12.1	build generator
LLVM / clang-cl	23.1.4	23.1.4	primary Windows compiler
MSVC (`cl.exe`)	19.44	19.44	optional Visual Studio-native Windows compiler
Clang (Linux)	23.1.4	23.1.4	primary Linux compiler
GCC	16.2.0	16.2.0	second compiler for portability checks
Python	3.11	3.13	documentation and notices generation
Git	2.40	—	clone and version control
Vulkan Loader	1.4	1.4	runtime on Windows and Linux
MoltenVK	1.2	—	Vulkan over Metal on macOS (or the Metal backend directly)
7-Zip	21	25.01	unpack the LZMA SDK (otherwise the built-in FbyLZ is used)
Xcode	26.4	—	macOS builds (AppleClang 21)

The language standard is C++26, so an older compiler will not work even when its CMake version matches.

Quick start with the setup scripts

The scripts live under `SetupScripts/<OS>/` and are split by OS and software

group. The one-click scripts install everything; the component scripts install

one group.

Group	Windows	macOS	Linux
C++ toolchain, CMake and source dependencies	`SetupScripts\Windows\SetupCpp.bat`	`./SetupScripts/macOS/SetupCpp.command`	`./SetupScripts/Linux/SetupCpp.sh`
7-Zip archiver	`SetupScripts\Windows\SetupArchiver.bat`	`./SetupScripts/macOS/SetupArchiver.command`	`./SetupScripts/Linux/SetupArchiver.sh`
Optional DLSS/FidelityFX/XeSS SDKs	`SetupScripts\Windows\Setup.bat -WithVendorSdks -AcceptVendorLicenses`	`./SetupScripts/macOS/Setup.command --with-vendor-sdks --accept-vendor-licenses`	`./SetupScripts/Linux/Setup.sh --with-vendor-sdks --accept-vendor-licenses`

The all-in-one entry points are `SetupScripts/Windows/Setup.bat`,

`SetupScripts/macOS/Setup.command` and `SetupScripts/Linux/Setup.sh`.

The C++ scripts install the compiler, CMake, Ninja, the Vulkan headers and the

pinned source dependencies. On Windows, `SetupScripts/Windows/SetupCpp.bat` installs the official LLVM

23.1.4 MSI and verifies the `clang-cl` version; the Unix scripts install the

distribution Clang (19+). The archiver scripts install 7-Zip. Vendor SDKs are

downloaded by the all-in-one Setup only after the corresponding licence flags;

nothing is part of the engine repository.

Useful flags: `--skip-system` leaves system packages alone (a CI image already

has the tools); `--force` re-downloads every C++ archive. The C++ script writes

`third_party/dependencies.lock` with the pinned versions and archive hashes.

At the end it prints a summary of the installed toolchains.

From a GitHub archive

1. Open the repository page and choose **Code > Download ZIP**.
2. Extract the archive to a directory without spaces or non-ASCII characters.
3. Run the host OS `SetupCpp` and `SetupArchiver` entry points from the table above.
4. Optionally pass the vendor licence flags to the host `Setup` entry point.
5. Configure and build:

```
cmake --preset dev-clang
cmake --build --preset dev-clang
ctest --preset dev-clang --output-on-failure
```

The archive contains no vendor SDK. The C++ script installs the toolchain and

materialises the pinned sources; the archiver script installs 7-Zip. The

upscaler scripts are only needed when a DLSS/FidelityFX backend is explicitly

enabled. Without them, FSR1, the FSR2-compatible core path and custom frame

generation build and run with runtime fallback.

Manual installation

Without the script the steps are the same: install a compiler, CMake, Ninja, the Vulkan headers, and lay out the third-party sources.

Windows

6. Install LLVM 23.1.4 (clang-cl) and add its `bin` directory to `PATH`.
7. Install Visual Studio Build Tools with the "C++ Build Tools" workload (Windows SDK and standard library headers). The MSVC compiler itself is not used by the clang-cl build.
8. Install CMake 3.31 and Ninja 1.12 (the Build Tools copy is fine).
9. Install the Vulkan SDK 1.4 — it provides `vulkan-1.dll` and `VULKAN_SDK`.
10. Install 7-Zip (optional).
11. Download the archives the script would fetch and unpack them into the same `third_party/` directories: SDL3 `release-3.4.18`, Vulkan-Headers `v1.4.366`, glslang `15.3.0`, miniaudio `0.11.23`, Box2D `v3.2.0`, RNNoise `0.1.1`, Monocypher `4.0.3`. HarfBuzz and SheenBidi need no download: they are already in the repository.

Linux

```
sudo apt install build-essential clang cmake ninja-build git pkg-config libvulkan-dev
vulkan-tools p7zip-full
```

Then lay out the same archives as on Windows. GCC 16.2 comes from the distribution or a separate build; Clang is enough for the primary build.

macOS

```
xcode-select --install
brew install cmake ninja git p7zip glslang vulkan-headers molten-vk llvm
```

Xcode Command Line Tools are mandatory: they provide `clang++`, the Metal frameworks and the SDK headers. The Metal backend needs no Vulkan SDK.

Building

From the repository root:

```
cmake -S . -B build-dev -G Ninja -DCMAKE_BUILD_TYPE=RelWithDebInfo -DFBY_BUILD_TESTS=ON
cmake --build build-dev
ctest --test-dir build-dev --output-on-failure
```

On Windows the same commands run inside a `vcvars64` environment; `SetupScripts/Windows/SetupWindows.ps1` prepares the dependencies and the toolchain. Main CMake options: `FBY_BUILD_EDITOR`, `FBY_BUILD_TESTS`, `FBY_BUILD_TOOLS`, `FBY_BUILD_SANDBOX`, `FBY_BUILD_SERVER`, `FBY_BUILD_LOBBY`, `FBY_WITH_ASAN`, `FBY_RHI_BACKEND` (`auto`, `vulkan`, `metal`, `null`).

The executables are grouped by purpose:

Directory	Contents
`build-dev/bin`	applications: `fby_editor`, `fby_sandbox`, `fby_server`, `fby_lobby`, platform plugins and `plugins/`
`build-dev/bin/tools`	console tools: `fby_fontc`, `fby_textc`, `fby_fempak`, `fby_scene_tool`, `fby_template_gen`, `fby_aotc`, `fby_loc_tool`, `fby_shaderc`
`build-dev/bin/tests`	test executables (`fby_*_test`) that `ctest` runs
`build-dev/bin/lib`	static libraries
`build-dev/bin/plugins`	native engine plugins

The `*.pdb` files next to the executables are debug symbols written by the compiler. They are only used by a debugger and are not needed to run anything; a release package does not have to ship them.

Code editor and IDE

The engine does not ship its own text editor. Development uses an external editor that understands the CMake project and `compile_commands.json`.

Recommended setup:

- **VS Code**;
- the **clangd** extension;
- the **CMake Tools** extension;
- **CodeLLDB** or `cppdbg` for debugging;
- RenderDoc for GPU debugging.

CLion also works: it understands CMake natively and can launch `fby_editor` and the tests. Visual Studio can be used as an IDE, but not as an MSVC build; see below.

VS Code setup

12. Install VS Code.
13. Install the extensions:

- `llvm-vs-code-extensions.vscode-clangd`;
- `ms-vscode.cmake-tools`;
- `vadimcn.vscode-lldb` or `ms-vscode.cpptools` for `cppdbg`.

14. Open the repository root (the folder that contains `CMakePresets.json`).
15. Disable the built-in IntelliSense; otherwise it competes with clangd:

```
{
  "C_Cpp.intelliSenseEngine": "disabled"
}
```

16. Select the `dev-clang` or `dev-clangcl` CMake preset.
17. Configure:

```
cmake --preset dev-clang
```

18. Build:

```
cmake --build --preset dev-clang
```

19. Run tests:

```
ctest --preset dev-clang
```

The repository already contains:

- `CMakePresets.json` — the presets;
- `.clangd` — compile-database path;
- `.vscode/settings.json` — clangd and CMake Tools settings;
- `.vscode/launch.json` — editor and `frame_composer` launch targets.

clangd

clangd must use the same Clang that builds the project:

```
{
  "clangd.arguments": [
    "--compile-commands-dir=${workspaceFolder}/build-audit",
    "--query-driver=C:/Users/<user>/AppData/Local/Programs/LLVM/bin/clang++.exe"
  ]
}
```

`--query-driver` lets clangd find the Windows SDK headers. If a different build directory is used, change `--compile-commands-dir` or `.clangd`.

Do not enable C/C++ IntelliSense and clangd at the same time: that produces two different diagnostic and navigation stacks.

Build presets

Preset	Compiler	Purpose
`dev-clang`	`clang++`	Debug, tests, `compile_commands.json`
`release-clang`	`clang++`	RelWithDebInfo, tests
`dev-clangcl`	`clang-cl`	Debug with the validated Windows toolchain
`audit-clang`	`clang++`	existing `build-audit` for clangd

Different compilers must use **different build directories**. Do not reconfigure `build-audit` from clang++ to clang-cl: the CMakeCache belongs to the old toolchain.

Debugging

For `cppdbg`, use gdb from MinGW/LLVM; CodeLLDB uses LLVM's lldb. Example:

```
{
  "name": "Editor (Debug)",
  "type": "cppdbg",
  "request": "launch",
  "program": "${workspaceFolder}/build-dev/bin/fby_editor.exe",
  "cwd": "${workspaceFolder}",
  "MIMode": "gdb",
  "miDebuggerPath": "gdb.exe"
}
```

Use RenderDoc for GPU debugging: attach it to `fby_editor.exe` and capture a frame without embedding a debugger in the editor.

Compiler selection

The 0.4 alpha line supports **Clang 23.1.4+** (clang-cl on Windows, clang++ on Unix), **AppleClang 21+**, **GCC 16.2.0+** and **MSVC 19.44+**. clang-cl is the reference toolchain; the others are supported alternatives, not an obligation. CI runs clang-cl, clang, GCC and AppleClang with the full test suite; MSVC 19.44+ builds the same tree and is supported for local development, but it is not part of the CI gate.

- **Windows, primary:** clang-cl. It is Clang with the MSVC ABI, the Windows SDK and the MSVC standard library, and it works with the Visual Studio debugger and PDBs.
- **Windows, optional:** MSVC (`cl.exe`). Choose it when the team standardizes on the Microsoft toolchain; CMake selects `/std:c++latest` and the MSVC warning set. RNNoise falls back to the spectral tier because its vendored C sources use variable-length arrays.
- **Linux, primary:** clang++. GCC 16.2.0 is the second compiler for portability checks.
- **macOS:** AppleClang 21+ from Xcode 26.4 with the Metal backend.
- **One compiler on every platform:** Clang (clang-cl on Windows, clang++ elsewhere). It is the only toolchain that covers all three desktop platforms with one frontend.

The full matrix and the per-OS presets are in [build-workflow.md](../../design/build-workflow.md).

Running the editor

The editor opens the project from its **current working directory**: it looks for `project.femproj`, mounts `assets/` as the virtual file system, writes project editor settings to `editor.ini` next to `project.femproj`, and keeps the ImGui layout in `Saved/Config/editor.ini`. Start it from the project root (the repository root for the demo project).

```
cd %USERPROFILE%\FemboyEngine
build-dev\bin\fby_editor.exe
```

Linux and macOS:

```
cd /path/to/project
./build-dev/bin/fby_editor
```

Everything the engine writes at runtime goes to `Saved/` in the working directory, so the project directory stays clean:

Path	Contents
`Saved/Logs/<category>/<category>-YYYYMMDD-HHMMSS.log`	the full log of a run: `engine`, `editor`, `sandbox`, `server`, `lobby`. The 20 newest files per category are kept and older ones are removed automatically
`Saved/Crashes/<YYYYMMDD-HHMMSS>-<pid>/report.txt`	crash report: time, application, pid, reason, address and, on Linux and macOS, a backtrace
`Saved/Crashes/<...>/crash.dmp`	Windows minidump for a debugger
`Saved/Config/editor.ini`	editor settings and panel layout
`Saved/Cache/`	cache of intermediate data

Environment variables: `FBY_STORAGE_DIR` moves the `Saved/` root (for example to a disk with space) and `FBY_LOG_FILE=0` disables the log file while the console keeps working.

Files that must be present

Path	Required	Purpose
`build-dev/bin/fby_editor`	yes	the editor itself
`assets/`	yes	fonts, localization, scenes, atlases, project configs
`assets/fonts/IBMPlexSans-Regular.ttf`, `...-Bold.ttf`, `...-Italic.ttf`, `...-BoldItalic.ttf`, `IBMPlexMono-Regular.ttf`	yes	the font family used by the editor and by scene text
`assets/fonts/IBMPlex.family.toml`	yes	family manifest with the five style slots
`assets/fonts/Inter.fbyfont`	yes	pre-baked atlas used by the legacy UI path
`assets/loc/ui.fbin`	yes	interface localization table
`assets/scenes/*.toml`	yes	project scenes
`project.femproj`	yes	project description: default scene, simulation, render, accessibility
`build-dev/`	yes	the CMake build tree: executables and build results live there
`build-dev/bin/fby_sandbox`	no	second process for networked Play-in-Editor
`Saved/`	created	logs, crash reports, caches and editor settings
`Saved/Config/editor.ini`	created	panel layout and editor settings for this machine
`build-dev/bin/plugins/`	no	native engine plugins
Vulkan Loader installed on the system	yes	window and GPU rendering on Windows and Linux
`Projects/`	no	default location for new projects

Selecting a project

A project is a directory with a `project.femproj` file and `assets/`, `configs/` subdirectories. New projects are created in `Projects/` inside the engine root; an existing one is opened from the editor dashboard by the path to `project.femproj` or to the project folder. There is no command-line argument for the project: the working directory decides, which is why a "project shortcut" is a shortcut with a different working directory.

Development tools

Besides the editor, `build-dev/bin` contains separate executables. All of them are console programs that read files and need no GUI.

Tool	Purpose	Files it needs
`fby_editor`	editor: scenes, assets, PIE, text preview	`assets/`, `project.femproj`
`fby_sandbox`	sandbox and Play-in-Editor client	`assets/`
`fby_server`	dedicated server	project directory if configured
`fby_lobby`	lobby and matchmaking service	none
`fby_fontc`	build a `.fbyfont` from a TTF/OTF	the input TTF
`fby_textc`	compile marked-up text into `.fbin`	the input `.toml` text
`fby_fempak`	pack assets into a `.fempak`	the asset directory
`fby_scene_tool`	validate and convert scenes	the scene `.toml`
`fby_template_gen`	generate a new project skeleton	destination directory
`fby_aotc`	nativize scripts (AOT)	project with bytecode
`fby_loc_tool`	validate and build localization tables	localization `.toml`
`fby_shaderc`	compile shaders to SPIR-V	`.vert`/`.frag`; the build calls it automatically

The tools live in `build-dev/bin/tools`; the test executables in `build-dev/bin/tests`.

Build and test through the CMake presets; dependency setup lives in `SetupScripts/<OS>/*`.

Shortcuts for fast launching

The working directory is the only thing that distinguishes "launching the engine" from "launching a project", so a shortcut must always set it explicitly.

Windows

20. Right-click `build-dev\bin\fby_editor.exe` → Send to → Desktop (create shortcut).
21. Open the shortcut properties: "Start in" = project root (for example `%USERPROFILE%\FemboyEngine`), "Target" = full path to `fby_editor.exe`.
22. Rename it to "FemboyEngine" and drag it to the taskbar or the Start menu.
23. For a specific project, create a second shortcut with the same target and "Start in" = the project folder. That is the project selection without a dialog.

macOS

24. Open the project folder in Finder.
25. Drag `fby_editor` (or the `.app` bundle, if built) to the Dock while holding `Command`.
26. For a specific project create a `.command` file with two lines: `cd /path/to/project` and `/path/to/fby_editor`; Finder runs it on double-click and the alias can live in the Dock.
27. `SetupScripts/macOS/Setup.command` starts the same way — by double-click.

Linux

Create `~/.local/share/applications/fby-editor.desktop`:

```
[Desktop Entry]
Type=Application
Name=FemboyEngine
Exec=/path/to/fby_editor %f
Path=/path/to/project
Icon=/path/to/Logo.png
Terminal=false
Categories=Development;
```

Then pin it to the dock or the panel. A second `.desktop` file with a different `Path` gives a shortcut to another project.

Verifying the toolchain

The exact versions can be checked with:

```
cmake --version
ninja --version
```

On macOS, `xcodebuild -version` reports the Xcode and SDK versions.

Troubleshooting

Symptom	Cause	Fix
The editor shows no assets	it was started outside the project root	start it from the directory with `assets/` and `project.femproj`
Vulkan error at startup	the system has no Vulkan Loader	install the Vulkan Runtime or build with `-DFBY_RHI_BACKEND=null`
The editor or a game closed without a message	the process crashed	open `Saved/Crashes/` in the working directory: `report.txt` and `crash.dmp`
A run log is needed for a report	logs are written to a file now	take the newest file from `Saved/Logs/<category>/`
`glslang` missing during the build	`third_party/glslang-src` is missing	run the setup script again
No fonts in the editor	the TTFs or the family manifest are missing	restore `assets/fonts/` from the repository
CMake reports a missing harfbuzz or SheenBidi	the `third_party/` directory was damaged or removed	restore `third_party/harfbuzz-download/` and `third_party/sheenbidi-download/` from the repository
The Windows build fails on	the Windows SDK is missing	install Build Tools with the C++

headers		workload